Basics of SQL

Programmazione Avanzata 2026-27

Outline

- Structured Query Language (SQL)
  - DDL (Data Definition Language)
  - DML (Data Manipulation Language)
- Query
- **`SELECT`** clause
- **`WHERE`** clause
- Sorting (**`ORDER BY`**)
- Join
- Aggregate functions (**`COUNT`**, **`SUM`**, **`AVG`**, **`MAX`**, **`MIN`**)
- Grouping (**`GROUP BY`** and **`HAVING`**)
- Nested queries/Subqueries

Structured Query Language (SQL)

- SQL is a language used to define the structure of a relational database and to access and modify its data
- SQL supports:
  - Interactive operations
  - Embedded operations: SQL statements are included within a host language (e.g., in Python) and distinguished from its instructions through appropriate syntactic constructs

Structured Query Language (SQL)

- SQL is a set-oriented language: operations and results involve sets of data
- It operates at a higher level of abstraction than other programming languages
  - It specifies what to do, rather than how to do it
  - For instance, a **`SELECT`** statement does not specify how data should be accessed, but describes the data to be retrieved

Structured Query Language (SQL)

- SQL is a language for managing relational databases
  - DDL (Data Definition Language) defines the structure of a database (i.e., its schema) using operators like **`CREATE TABLE`**, **`ALTER TABLE`** and **`DROP TABLE`**
  - DML (Data Manipulation Language) enables access and manipulation of data stored in the database using operators like, **`INSERT INTO`**, **`SELECT`**, **`UPDATE`** and **`DELETE`**

Database

- Relational model:
  - Product table **`P`**
  - Supplier table **`S`**
  - Supply table **`SP`**, relating products to suppliers that supply them
- Primary key: tuple identifier
  - For products: **`P#`**
  - For suppliers: **`S#`**
  - For supplies: (**`S#`**, **`P#`**)
- Foreign keys ensure that every supply recorded in **`SP`** refers to an existing supplier and an existing product

Creating the database

```
CREATE TABLE S
(
    `S#` CHAR(5) NOT NULL,
    SNAME CHAR(20) NOT NULL,
    STATUS SMALLINT NOT NULL,
    CITY CHAR(15) NOT NULL,
    PRIMARY KEY (`S#`)
);
```

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

Creating the database

```
CREATE TABLE P
(
    `P#` CHAR(6) NOT NULL,
    PNAME CHAR(20) NOT NULL,
    COLOR CHAR(6) NOT NULL,
    WEIGHT SMALLINT NOT NULL,
    CITY CHAR(15) NOT NULL,
    PRIMARY KEY (`P#`)
);
```

P#	PNAME	COLOR	WEIGHT	CITY
P1	Nut	Red	12	London
P2	Bolt	Green	17	Paris
P3	Screw	Blue	17	Rome
P4	Screw	Red	14	London
P5	Cam	Blue	12	Paris
P6	Cog	Red	19	London

Creating the database

```
CREATE TABLE SP
(
    `S#` CHAR(5) NOT NULL,
    `P#` CHAR(6) NOT NULL,
   QTY INTEGER NOT NULL,
    PRIMARY KEY (`S#`, `P#`),
    FOREIGN KEY (`S#`) REFERENCES S (`S#`),
    FOREIGN KEY (`P#`) REFERENCES P (`P#`)
);
```

S#	P#	QTY
S1	P1	300
S1	P2	200
S1	P3	400
S1	P4	200
S1	P5	100
S1	P6	100
S2	P1	300
S2	P2	400
S3	P2	200
S4	P2	200
S4	P4	300
S4	P5	400

Suppliers-products database

S

S#	SNAME	STATUS	CITY
S1	Smith	20	London
S2	Jones	10	Paris
S3	Blake	30	Paris
S4	Clark	20	London
S5	Adams	30	Athens

P

P#	PNAME	COLOR	WEIGHT	CITY
P1	Nut	Red	12	London
P2	Bolt	Green	17	Paris
P3	Screw	Blue	17	Rome
P4	Screw	Red	14	London
P5	Cam	Blue	12	Paris
P6	Cog	Red	19	London

SP

S#	P#	QTY
S1	P1	300
S1	P2	200
S1	P3	400
S1	P4	200
S1	P5	100
S1	P6	100
S2	P1	300
S2	P2	400
S3	P2	200
S4	P2	200
S4	P4	300
S4	P5	400

Retrieving data

- The general structure of a query to retrieve data from the database is as follows:

```
SELECT [DISTINCT] column(s)
FROM table(s)
[WHERE conditions]
[GROUP BY column(s)]
[HAVING conditions]
[ORDER BY column(s)];
```

Query

- Selects the attributes specified in the target list for the rows that satisfy the condition in the **`WHERE`** clause, from the Cartesian product of the tables listed in the **`FROM`** clause
- The result of a query is a table, and the target list specifies the attributes (columns) included in the schema of that table
- The special character **`*`** can be used as the target list to select all attributes from the tables listed in the **`FROM`** clause

SELECT clause

- Retrieving all information in table **S**:

```
SELECT *
FROM S
```

- Retrieving attributes **NAME** and **STATUS**:

```
SELECT NAME, STATUS
FROM S
```

- Fully qualified identifiers, using table name followed by .:

```
SELECT S.S#, S.NAME, S.STATUS, S.CITY
FROM S
```

SELECT with expressions

- Retrieving information with expressions:

```
SELECT P.P#, P.WEIGHT*454 AS Grams
FROM P
```

P#	Grams
P1	5448
P2	7718
P3	7718
P4	6356
P5	5448
P6	8626

Example

- Duplicates are not removed

```
SELECT P#
FROM SP
```

P#
P1
P2
P3
P4
P5
P6
P1
P2
P2
P2
P4
P5

SELECT with DISTINCT

- Removal of duplicates

```
SELECT DISTINCT P#
FROM SP
```

P#
P1
P2
P3
P4
P5
P6

WHERE clause

- The **`WHERE`** clause takes as its argument a Boolean expression built by combining simple predicates (using the operators **`=`**, **`<`**, **`>`**, **`<>`**, **`<=`**, and **`>=`**) with the logical operators **`AND`**, **`OR`**, and **`NOT`**
- A simple expression compares the value of an attribute with another attribute value or with a constant

Example

- Retrieve the codes and status of suppliers located in Paris

```
SELECT S#, STATUS
FROM S
WHERE CITY = 'Paris'
```

S#	STATUS
S2	10
S3	30

Example

- Qualified retrieval: retrieve the numbers of suppliers in Paris whose status is greater than 20

```
SELECT S#
FROM S
WHERE CITY = 'Paris'
AND STATUS > 20
```

S#
S3

Sorting: ORDER BY

- A table consists of an unordered set of tuples
- To sort the tuples, the **`ORDER BY`** clause must be used
- Syntax:

```
ORDER BY sort_attribute [ASC | DESC]
{, sort_attribute [ASC | DESC]}
```

- Tuples are sorted by the first specified attribute according to the chosen order: when values are equal, the next attribute and its corresponding sort order are considered

Example

- Retrieve the codes and status of suppliers in Paris, sorted by status in descending order:

```
SELECT S#, STATUS FROM S
WHERE CITY = 'Paris'
ORDER BY STATUS DESC
```

S#	STATUS
S3	30
S2	10

Example

- The column used for sorting can be identified by its ordinal position in the target list

```
SELECT P.P#, 'Weight in grams=', P.WEIGHT * 454
FROM P
ORDER BY 3, P#
```

P#		
P1	Weight in grams =	5448
P5	Weight in grams =	5448
P4	Weight in grams =	6356
P2	Weight in grams =	7718
P3	Weight in grams =	7718
P6	Weight in grams =	8626

Approximate text retrieval

- For approximate text retrieval, the **`LIKE`** construct is used
- Syntax:

```
column_name LIKE char_string_constant
```

- In the **`char_string_constant`**:
  - The `_` character matches any single character
  - The `%` character matches any sequence of characters

Example

- Retrieve information about products whose names begin with the letter `'c'`:

```
SELECT P.* FROM P
WHERE P.PNAME LIKE 'c%'
```

P#	PNAME	COLOR	WEIGHT	CITY
P5	Cam	Blue	12	Paris
P6	Cog	Red	19	London

Approximate text retrieval

- **ADDRESS** contains the string **'Berkeley'**

```
ADDRESS LIKE '%Berkeley%'
```

- **S#** consists of exactly three characters, the first of which is **'S'**

```
S# LIKE 'S__'
```

- **PNAME** contains at least four characters, with **'c'** as the fourth character from the end

```
PNAME LIKE '%c___'
```

- **CITY** does not contain an **'E'**

```
CITY NOT LIKE '%E%'
```

NULL values

- Comparing values with **`NULL`** never produces a valid result, regardless of the comparison operator used
- Comparison predicate for **`NULL`** values:

```
column-name IS [NOT] NULL
```

Example

- Retrieve the codes of suppliers whose status is greater than 25 (assuming that supplier **S5** has a **NULL** status)

```
SELECT S# FROM S
WHERE STATUS > 25
```

S#
S3

Example

- Retrieve the codes of all suppliers whose status is NULL

```
SELECT S# FROM S
WHERE STATUS IS NULL
```

Join

- Join is operation used to combine rows from two (or more) tables
  - It is also the generic term used to refer to combining data from two (or more) tables, regardless of the condition used

Join

- Retrieve all supplier codes paired so that the part and the supplier are located in the same city
- Retrieve all supplier–product combinations in which the supplier's city follows the product's city alphabetically
- Retrieve all supplier–part combinations in which the suppliers and the corresponding parts are located in the same city, excluding suppliers whose status is 20
- Retrieve the codes and cities of suppliers that supply product **`P2`**

Join

- Retrieve all pairs of city names such that a supplier from the first city supplies a part stored in the second city
- Retrieve all cities in which there is a supply involving a supplier and a product from the same city
- Retrieve all pairs of suppliers located in the same city

Join

- There are multiple types of join operations, which differ in the condition used:
  - Cross join: Cartesian product; every row of one table is combined with every row of the other
  - Theta join: join using any comparison operator
  - Equi-join: theta join using only equality
  - Natural join: equi-join automatically performed on all columns having the same names; duplicate join columns are removed
  - Outer join: also retains rows for which no matching row exists in the other table

Cross join (Cartesian product)

- The resulting table consists of all possible rows *r*, where each *r* is formed by concatenating one row from the first table, one from the second, and so on, up to the n-th table

Cross join (Cartesian product)

S#	SNAME	STATUS	S.CITY	P#	PNAME	COLOR	WEIGHT	P.CITY
S1	Smith	20	London	P1	Nut	Red	12	London
S1	Smith	20	London	P2	Bold	Green	17	Paris
S1	Smith	20	London	P3	Screw	Blue	17	Rome
S1	Smith	20	London	P4	Screw	Red	14	London
S1	Smith	20	London	P5	Cam	Blue	12	Paris
S1	Smith	20	London	P6	Cog	Red	19	London
S2	Jones	10	Paris	P1	Nut	Red	12	London
…	…	…	…	…	…	…	…	…
…	…	…	…	…	…	…	…	…
S5	Adams	30	Athens	P6	Cog	Red	19	London

Theta join

- Retrieve all supplier–product combinations in which the supplier's city follows the product's city alphabetically:

```
SELECT S.*, P.*
FROM S, P
WHERE S.CITY > P.CITY
```

S#	SNAME	STATUS	S.CITY	P#	PNAME	COLOR	WEIGHT	P.CITY
S2	Jones	10	Paris	P1	Nut	Red	12	London
S2	Jones	10	Paris	P4	Screw	Red	14	London
S2	Jones	10	Paris	P6	Cog	Red	19	London
S3	Blake	30	Paris	P1	Nut	Red	12	London
S3	Blake	30	Paris	P4	Screw	Red	14	London
S3	Blake	30	Paris	P6	Cog	Red	19	London

Equi-join

- Retrieve all supplier and part code combinations in which the part and the supplier are located in the same city:

```
SELECT S.S#, P.P#
FROM S, P
WHERE S.CITY = P.CITY
```

S#	P#
S1	P1
S1	P4
S1	P6
S2	P2
S2	P5
S3	P2
S3	P5
S4	P1
S4	P4
S4	P6

Join with multiple predicates

- Retrieve all supplier–part combinations in which the suppliers and the corresponding parts are located in the same city, excluding suppliers whose status is 20:

```
SELECT S.*, P.*
FROM S, P
WHERE S.CITY = P.CITY AND
S.STATUS <> 20
```

S#	SNAME	STATUS	S.CITY	P#	PNAME	COLOR	WEIGHT	P.CITY
S2	Jones	10	Paris	P2	Bolt	Green	17	Paris
S2	Jones	10	Paris	P5	Cam	Blue	12	Paris
S3	Blake	30	Paris	P2	Bolt	Green	17	Paris
S3	Blake	30	Paris	P5	Cam	Blue	12	Paris

Equi-join between referenced tables

- When performing an equijoin between referenced tables, the **`WHERE`** clause must specify the equality condition between the columns that relate the tables

Example

- Retrieve the codes and cities of suppliers that supply product **P2**:

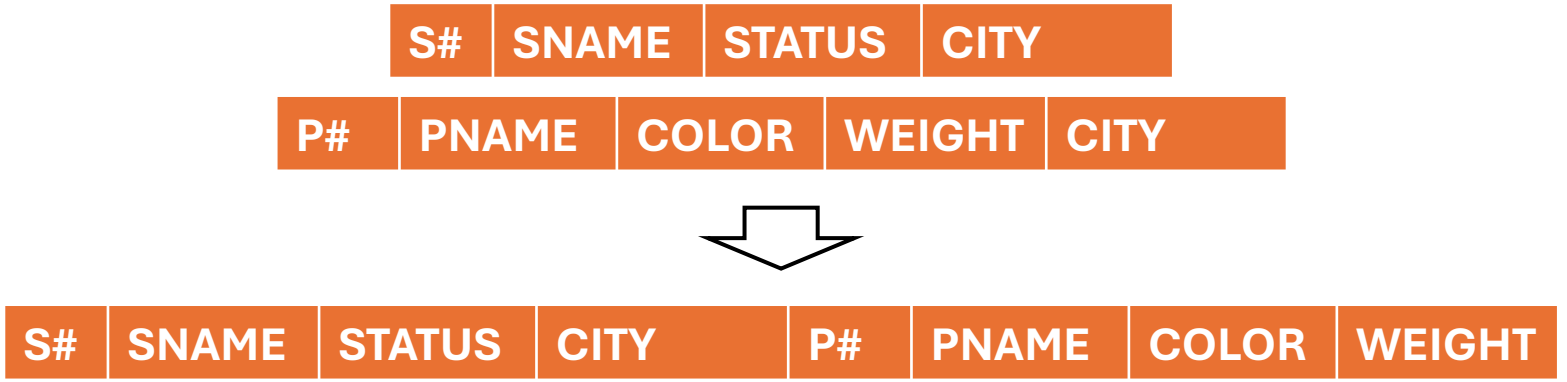
```
SELECT S.S#, S.CITY
FROM SP, S
WHERE SP.S# = S.S#
AND SP.P# = 'P2'
```

S#	CITY
S1	London
S2	Paris
S3	Paris
S4	London

Natural join

- It results in the elimination of one of the two **CITY** columns

```
SELECT S#, SNAME, STATUS, S.CITY, P#,
PNAME, COLOR, WEIGHT
FROM S, P
WHERE S.CITY = P.CITY
```



Joining three tables

- Retrieve all pairs of city names such that a supplier from the first city supplies a part stored in the second city:

```
SELECT DISTINCT S.CITY, P.CITY
FROM S, SP, P
WHERE S.S# = SP.S#
AND SP.P# = P.P#
```

S.CITY	P.CITY
London	London
London	Paris
London	Rome
Paris	London
Paris	Paris

Joining three tables with a condition

- Retrieve all cities in which there is a supply involving a supplier and a product located in the same city:

```
SELECT DISTINCT S.CITY
FROM S, SP, P
WHERE S.S# = SP.S#
AND SP.P# = P.P#
AND S.CITY = P.CITY
```

S.CITY
London
Paris

Aliases

- In SQL, an alternative name (called an alias) can be assigned to a table appearing in the **`FROM`** clause
- The alias is used to refer to the table within the query: short aliases avoid repeatedly writing the table's full name
- Aliases make it possible to refer to multiple instances of the same table: each alias introduces a table variable whose value is the content of the table to which the alias refers

Joining a table with itself

- Retrieve all pairs of suppliers located in the same city:

```
SELECT FIRST.S#, SECOND.S#
FROM S FIRST, S SECOND
WHERE FIRST.CITY = SECOND.CITY AND
FIRST.S# < SECOND.S#
```

S#	S#
S1	S4
S2	S3

Aggregate functions

- Aggregate functions make it possible to evaluate properties that depend on sets of tuples:
  - **`COUNT`**
  - **`SUM`**
  - **`AVG`**
  - **`MAX`**
  - **`MIN`**
- They appear in the target list

COUNT

- Syntax:

```
COUNT(<* | [DISTINCT | ALL] attribute_list)
```

- Available options:
  - **`COUNT(*)`** returns the number of rows
  - **`COUNT(DISTINCT attribute_list)`** returns the number of distinct values of the attributes specified in **`attribute_list`**
  - **`COUNT(ALL attribute_list)`** returns the number of rows containing values not **`NULL`** for the attributes specified in **`attribute_list`**

Examples

- Count the tuples in **S**:

```
SELECT COUNT(*) FROM S
```

- Count the number of suppliers that supply at least one product:

```
SELECT COUNT(DISTINCT S#)
FROM SP
```

- Count the supplies of product **P2**:

```
SELECT COUNT(*) FROM SP
WHERE P# = 'P2'
```

SUM, AVG, MAX, MIN

- Syntax:

```
<SUM | MAX | MIN | AVG>
([DISTINCT | ALL] argument)
```

- These functions operate on the rows that satisfy the condition in the **`WHERE`** clause; the argument can be an attribute or an expression
- Their meanings are:
  - **`SUM`**: returns the sum of the values of the specified attribute
  - **`MAX`** and **`MIN`**: return the maximum and minimum values, respectively
  - **`AVG`**: returns the average of the values of the specified attribute

SUM, AVG, MAX, MIN

- **`SUM`** and **`AVG`** accept only expressions representing numeric values or time intervals as arguments
- **`MAX`** and **`MIN`** require arguments for which an ordering is defined and can be applied to character strings, numeric values, or points in time
- **`DISTINCT`** eliminates duplicate values
- **`ALL`** ignores **`NULL`** values

Example

- Retrieve the total quantity of product **P2** supplied:

```
SELECT SUM(QTY) FROM SP
WHERE P# = 'P2'
```

1000

GROUP BY

- The **`GROUP BY`** clause divides a table into subsets, that is, groups of tuples
- Syntax:

```
GROUP BY attribute_list
```

- The query groups together rows that have the same values for the attributes specified in the **`GROUP BY`** clause

GROUP BY and aggregate functions

- It is often necessary to apply an aggregate function separately to subsets of rows:
  - The **`GROUP BY`** clause groups rows into subsets
  - The aggregate function specified in the target list is applied to each subset

Example

- Calculate the total quantity supplied for each product

```
SELECT P#, SUM(QTY)
FROM SP
GROUP BY P#
```

P#	SUM(QTY)
P1	600
P2	1000
P3	400
P4	500
P5	500
P6	100

Target list and GROUP BY

- SQL syntax requires that, in a query using the **`GROUP BY`** clause, any non-aggregated attributes appearing in the **`SELECT`** list must be included among the attributes specified in the **`GROUP BY`** clause: each tuple in a group must have the same value for these attributes

Example (incorrect)

- Consider:

```
SELECT S#
FROM SP
GROUP BY P#
```

- This query is incorrect because each group, whose rows share the same **P#**, does not necessarily correspond to a single supplier (**S#**)

Example (correct)

- For each product, display its code, the city where it is stored, and the total quantity supplied

```
SELECT P.P#, P.CITY, SUM(SP.QTY)
FROM SP, P
WHERE SP.P# = P.P#
GROUP BY P.P#, P.CITY
```

P#	CITY	SUM(SP.QTY)
P1	London	600
P2	Paris	1000
P3	Rome	400
P4	London	500
P5	Paris	500
P6	London	100

Predicates on groups

- The **`HAVING`** predicate makes it possible to apply selection criteria to groups of tuples

Example

- Retrieve the codes of parts supplied by more than one supplier:

```
SELECT P# FROM SP
GROUP BY P#
HAVING COUNT(*) > 1
```

P#
P1
P2
P4
P5

HAVING vs WHERE

- The **`WHERE`** clause is applied to each tuple, determining whether that tuple is selected or excluded
- The **`HAVING`** clause is applied to each group, determining whether that group of tuples is selected or excluded.
- The **`HAVING`** clause takes a Boolean expression composed of simple predicates: these predicates normally compare the result of an aggregate function with an expression

Example

- For each supplier, display its code and the total quantity of products supplied from London, provided that the total is greater than 500:

```
SELECT S#, SUM(QTY)
FROM P, SP
WHERE P.P# = SP.P#
AND P.CITY = 'London'
GROUP BY S#
HAVING SUM(QTY) > 500
```

S#	SUM(QTY)
S1	600
S2	300
S4	300

Example (correct but inefficient)

- For each supplier located in London, find the total quantity of products supplied from London:

```
SELECT S#, SUM(QTY)
FROM P, SP, S
WHERE P.P# = SP.P#
AND SP.S# = S.S#
AND P.CITY = 'London'
GROUP BY S#, S.CITY
HAVING S.CITY = 'London'
```

S#	SUM(QTY)
S1	600
S4	300

Example (correct, efficient)

- For each supplier located in London, find the total quantity of products supplied from London:

```
SELECT S#, SUM(QTY)
FROM P, SP, S
WHERE P.P# = SP.P#
AND SP.S# = S.S#
AND P.CITY = 'London'
AND S.CITY = 'London'
GROUP BY S#
```

S#	SUM(QTY)
S1	600
S4	300

Nested queries/Subqueries

- SQL allows an attribute to be compared with the result of a query
- Two forms are possible:
  - Subquery
  - Existential quantification
- Syntax:

```
WHERE <attribute comparison_operator
| [NOT] EXISTS (select_clause)>
```

Example

- Retrieve the codes of suppliers whose status is lower than the maximum value currently present in the table:

```
SELECT S# FROM S
WHERE STATUS <
    (SELECT MAX(STATUS) FROM S);
```

S#
S1
S2
S4

Example

- Retrieve the code of each product whose weight is greater than the average weight of all products:

```
SELECT P# FROM P
WHERE WEIGHT >
    (SELECT AVG(WEIGHT) FROM P)
```

P#
P2
P3
P6

Example

- Retrieve the codes of suppliers operating in the same city as supplier **S1**:

```
SELECT S# FROM S
WHERE CITY =
    (SELECT CITY FROM S WHERE S# = 'S1');
```

S#
S1
S4

Example

- Retrieve the codes of suppliers for which at least one supplier has a higher status:

```
SELECT S1.S#
FROM S AS S1
WHERE EXISTS (
    SELECT *
    FROM S AS S2
    WHERE S2.STATUS > S1.STATUS
);
```

S#
S1
S2
S4

Nested queries/Subqueries

- SQL provides two additional operators:
  - **`IN`**
  - **`NOT IN`**

Example

- Retrieve the names of suppliers that supply product **P2**:

```
SELECT SNAME FROM S
WHERE S# IN
    (SELECT S# FROM SP WHERE P# = 'P2');
```

SNAME
Smith
Jones
Blake
Clark

Example (with join)

- The previous query is equivalent to the following join:

```
SELECT S.SNAME FROM S, SP
WHERE S.S# = SP.S# AND SP.P# = 'P2'
```

SNAME
Smith
Jones
Blake
Clark

Example

- Retrieve the names of suppliers that do not supply product **P2**:

```
SELECT SNAME FROM S
WHERE S# NOT IN
    (SELECT S# FROM SP WHERE P# = 'P2');
```

Example

- Retrieve the names of suppliers that supply at least one red product:

```
SELECT SNAME FROM S
WHERE S# IN
    (SELECT S# FROM SP
    WHERE P# IN
        (SELECT P# FROM P
        WHERE COLOR = 'Red'));
```

SNAME
Smith
Jones
Clark

Example (with join)

- Retrieve the names of suppliers that supply at least one red product.

```
SELECT SNAME
FROM S, SP, P
WHERE S.S# = SP.S#
AND P.P# = SP.P#
AND P.COLOR = 'Red'
```

Modifying data

- The language also provide operations for inserting, deleting, and updating one or more tuples in a relation
- The operations may be based on a condition that also involve other relations

INSERT INTO operator

- Syntax:

```
INSERT INTO table_name [attribute_list]
< VALUES (list_of_values) | select_clause>
```

- Two formats are possible:

```
INSERT INTO table_name [attribute_list
VALUES (list_of_values)
```

```
INSERT INTO table_name [attribute_list]
select_clause
```

INSERT INTO operator

- The order of the attributes (if specified) and their corresponding values is significant
  - The two lists must contain the same number of elements
  - If the attribute list is omitted, all attributes of the relation are considered in the order in which they were defined
  - If the attribute list does not include all attributes of the relation, the remaining attributes are assigned either a **`NULL`** value, if permitted, or their default value

Example

- Insert a new tuple for product **P7**, stored in Athens and weighing 24:

```
INSERT INTO P (P#, CITY, WEIGHT)
VALUES ('P7', 'Athens', 24);
```

- Note that **PNAME** and **COLOR** are initialized to **NULL**

Example

- Insert product **P8** setting **PNAME** to **'Sprocket'**, **COLOR** to **'Pink'**, **WEIGHT** to **12** and **CITY** to **'Nice'**:

```
INSERT INTO P
VALUES ('P8', 'Sprocket', 'Pink', 12, 'Nice');
```

- Omitting the column list is equivalent to specifying all columns in the order in which they were defined in the table

Populating the database

```
INSERT INTO S (`S#`, SNAME, STATUS, CITY) VALUES
('S1', 'Smith', 20, 'London'),
('S2', 'Jones', 10, 'Paris'),
('S3', 'Blake', 30, 'Paris'),
('S4', 'Clark', 20, 'London'),
('S5', 'Adams', 30, 'Athens');
```

Populating the database

```
INSERT INTO P (`P#`, PNAME, COLOR, WEIGHT, CITY) VALUES
('P1', 'Nut', 'Red', 12, 'London'),
('P2', 'Bolt', 'Green', 17, 'Paris'),
('P3', 'Screw', 'Blue', 17, 'Rome'),
('P4', 'Screw', 'Red', 14, 'London'),
('P5', 'Cam', 'Blue', 12, 'Paris'),
('P6', 'Cog', 'Red', 19, 'London');
```

Populating the database

```
INSERT INTO SP (`S#`, `P#`, QTY) VALUES
('S1', 'P1', 300),
('S1', 'P2', 200),
('S1', 'P3', 400),
('S1', 'P4', 200),
('S1', 'P5', 100),
('S1', 'P6', 100),
('S2', 'P1', 300),
('S2', 'P2', 400),
('S3', 'P2', 200),
('S4', 'P2', 200),
('S4', 'P4', 300),
('S4', 'P5', 400);
```

DELETE operator

- Syntax:

```
DELETE FROM table_name
[WHERE conditions]
```

- Deletes from **`table_name`** all records that satisfy the conditions

Example

- Delete all suppliers located in Madrid:

```
DELETE FROM S
WHERE CITY = 'Madrid';
```

Example

- Delete all supplies made by suppliers located in London:

```
DELETE FROM SP
WHERE 'London' =
    (SELECT CITY FROM S WHERE S.S# = SP.S#);
```

UPDATE operator

- Syntax:

```
UPDATE table_name
SET attribute = <expression |
query | NULL | DEFAULT>
{, attribute = <expression | query
| NULL | DEFAULT>}
[WHERE conditions]
```

- Assigns new values to one or more attributes of the rows in **`table_name`** that satisfy the optional conditions

UPDATE operator

- The new value can be:
  - The result of evaluating an expression involving the table's attributes
  - The result of a generic query
  - The **`NULL`** value
  - The default value defined for the **`attribute`**'s domain

Example

- Update product **P1** setting **COLOR** to **'Yellow'**, incrementing **WEIGHT** by **12**, and setting **CITY** to **NULL**

```
UPDATE P
SET COLOR = 'Yellow' ,
WEIGHT=WEIGHT+12,
CITY = NULL,
WHERE P# = 'P1';
```

Example

- Set the quantity supplied to 0 for all suppliers located in London:

```
UPDATE SP
SET QTY = 0
WHERE 'London' =
    (SELECT CITY FROM S WHERE S.S# = SP.S#);
```